LangChain Models Component

What is the Models Component in LangChain?

Suppose you want to work with different AI models, such as:

- GPT by OpenAI
- Gemini by Google AI
- Claude by Anthropic
- Open-source models from Hugging Face

The problem is that each provider has its own API structure and coding style. The code used to interact with GPT is different from the code required for Gemini or Claude.

LangChain solves this problem by providing a **common interface**.

This means you can use a similar coding pattern to interact with different AI models without learning the unique implementation details of each provider.

In simple terms:

LangChain acts like a universal remote control for AI models.

Just as a universal remote can control different television brands, LangChain can interact with different AI models through a standardized interface.

Types of Models in LangChain

LangChain primarily works with two categories of models:

1. Language Models

Language models take text as input and generate text as output.

Example:

Input

What is the capital of India?

Output

New Delhi

These models are designed to answer questions, generate content, summarize text, and perform many other language-related tasks.

2. Embedding Models

Embedding models work differently.

Instead of generating text, they convert text into numerical representations called **vectors**.

Example:

Input

I love cricket

Output

[0.12, 0.45, -0.21, 0.88, ...]

This output is known as an **embedding vector**.

Why Are Embeddings Important?

Computers do not naturally understand the meaning of language.

To help computers understand semantic relationships, text is converted into vectors.

For example:

Virat Kohli
Cricket Player

These phrases have similar meanings, so their embedding vectors will be close to each other.

On the other hand:

Virat Kohli
Pizza Recipe

These phrases are unrelated, so their vectors will be far apart.

Embeddings are the foundation of:

- Semantic Search
- Recommendation Systems
- RAG (Retrieval-Augmented Generation)
- Document Search
- Similarity Matching

Types of Language Models

Language models can be divided into two major categories.

1. LLM (Large Language Model)

Traditional LLMs operate on plain text.

They take text as input and return text as output.

Example:

Input

Summarize this article

Output

This article discusses...

These models work with raw text and do not inherently understand conversational roles or chat history.

2. Chat Models

Chat Models are the most widely used models today.

Examples include:

- ChatGPT
- Gemini
- Claude

Instead of plain text, they operate on a sequence of messages.

Example:

```
[
   {"role": "system", "content": "You are a doctor"},
   {"role": "user", "content": "I have a fever"}
]
```

Roles are extremely important in chat models.

System Role

The system role defines the behavior and personality of the model.

Examples:

You are a doctor.

or

You are a Python expert.

User Role

The user role contains the user's request or question.

Example:

How can I reduce my fever?

Assistant Role

The assistant role stores previous responses from the AI and helps maintain conversation history.

Example:

Take proper rest and drink plenty of water.

Advantages of Chat Models

One of the biggest advantages is their ability to maintain context.

Example:

User

My name is Rahim.

Assistant

Nice to meet you.

User

What is my name?

Model Response

Your name is Rahim.

The model can answer correctly because previous messages are included in the conversation context.

What Are Closed-Source Models?

Closed-source models are proprietary models developed by companies.

You can access them through APIs, but you cannot view their training data, architecture, or model weights.

Examples:

- GPT
- Claude
- Gemini

Advantages

Closed-source models are generally:

- More powerful
- More accurate
- Better at reasoning
- Frequently updated and improved

Disadvantages

They require API usage fees.

Every request consumes tokens, and increased usage leads to higher costs.

What Are Open-Source Models?

Open-source models can be downloaded and run on your own machine or server.

Examples:

- TinyLlama
- Llama
- Mistral
- Gemma

Advantages

1. No API Costs

Once the model is downloaded, there are no recurring API charges.

2. Better Data Privacy

Your data remains on your own infrastructure and is not sent to external providers.

This is especially important for organizations handling sensitive information.

Disadvantages

- Requires your own hardware
- May require a GPU
- Often less powerful than top-tier closed-source models

What Is the Temperature Parameter?

Temperature controls the creativity and randomness of model responses.

Temperature = 0

Produces highly deterministic and consistent answers.

Example:

2 + 2 = ?

Output:

4

The answer will almost always be the same.

Temperature = 1

Produces moderately creative responses.

Temperature = 1.5+

Produces highly creative and random responses.

Example:

Write a short story.

The output may vary significantly each time.

When Should You Use Different Temperatures?

Coding

Temperature = 0

Accuracy is more important than creativity.

Mathematics

Temperature = 0

Deterministic answers are preferred.

Creative Writing

Temperature = 1 or higher

Creativity becomes more important than consistency.

What Are Max Completion Tokens?

The **Max Completion Tokens** parameter limits the maximum length of the model's response.

Example:

max_tokens = 100

The model will not generate more than 100 tokens.

Why Is It Useful?

Cost Control

Longer responses consume more tokens, which increases API costs.

Response Length Control

Useful when you need concise answers.

Example:

Answer in one sentence.

Document Similarity: The Most Important Practical Application

The final section demonstrates how embeddings can be used for document retrieval.

Imagine you have 1,000 documents stored in a knowledge base.

A user asks:

Tell me about Virat Kohli.

Sending all 1,000 documents to the model would be:

- Expensive
- Slow
- Inefficient

Instead, embeddings are used.

Step 1: Generate Embeddings for Documents

Document 1 → Vector
Document 2 → Vector
Document 3 → Vector

Step 2: Generate an Embedding for the User Query

Tell me about Virat Kohli
→ Vector

Step 3: Calculate Similarity Using Cosine Similarity

The system compares the query vector with all document vectors and finds the most relevant document.

What Is Cosine Similarity?

Cosine Similarity measures how similar two vectors are.

Typical values:

1 = Perfect Match
0 = No Similarity
-1 = Completely Opposite

Example 1

Virat Kohli
Indian Cricketer

Similarity:

0.95

Very similar.

Example 2

Virat Kohli
Pizza Recipe

Similarity:

0.08

Very different.

After finding the most relevant document, only that document (or a few top documents) is sent to the language model.

The model then generates an answer based on the retrieved information.

This is the core idea behind **RAG (Retrieval-Augmented Generation)**.

Summary

This lesson introduces the fundamental concepts of LangChain's Models Component:

1. How LangChain provides a common interface for different AI models.
2. The difference between Language Models and Embedding Models.

3. The distinction between LLMs and Chat Models.
4. The advantages and disadvantages of Open-Source and Closed-Source models.
5. How Temperature and Max Tokens control model behavior.
6. How Embeddings and Cosine Similarity enable document retrieval and RAG systems.

Understanding these concepts is essential before moving on to more advanced LangChain topics such as Prompts, Chains, Memory, RAG, and Agents.